



Miguel Angel Peralta Cano

Actividad

Actividad 9 - Informe final

Segundo semestre

Técnico Profesional en Servicios de Seguridad Informática

Materia:

PROGRAMACIÓN Y DESARROLLO DE SOFTWARE

Profesor:

José Luis Rodríguez Cortés

Chía - Cundinamarca

2025

Introducción

El presente informe detalla la documentación técnica y el diseño arquitectónico de la solución de software desarrollada para el caso de estudio de la empresa prestadora de servicios públicos UNIR S.A.. El objetivo principal de este proyecto es sistematizar el cálculo de tarifas de los servicios de agua, energía y gas, garantizando precisión matemática, validación de datos y la generación de un registro histórico de las transacciones.

Para lograr este propósito, el desarrollo del sistema se estructuró bajo los pilares de la Programación Orientada a Objetos (POO) y el patrón de diseño Modelo-Vista-Controlador (MVC), implementados a través del lenguaje Java. En este documento se exponen, en primer lugar, las historias de usuario que traducen los requerimientos del cliente en funcionalidades concretas. Posteriormente, se presenta la elaboración de los diagramas del Lenguaje Unificado de Modelado (UML) —incluyendo casos de uso, clases, objetos, secuencia y componentes— los cuales representan de manera gráfica y lógica el comportamiento, la estructura y la interacción del sistema. Esta documentación sirve como respaldo integral para garantizar que el producto final sea robusto, escalable y cumpla con las necesidades operativas de la organización.

B. Contenido

1. Descripción de la Arquitectura del Código

El sistema fue desarrollado en el lenguaje Java utilizando el patrón de arquitectura de software MVC (Modelo-Vista-Controlador). Esta estructura permite separar la lógica matemática de la interfaz gráfica, haciendo que el código sea mantenible y escalable.

El proyecto se dividió en tres secciones principales (paquetes):

Paquete com.unir.modelo (El Modelo): Contiene la lógica de negocio y aplica los pilares de la Programación Orientada a Objetos (POO). Se creó una clase abstracta padre llamada ServicioPublico (aplicando abstracción y encapsulamiento con atributos protected). De esta clase heredan las clases hijas Agua, Energía y Gas, las cuales utilizan el polimorfismo para sobrescribir el método calcularPago() asignando las tarifas específicas requeridas por el caso de estudio (\$730, \$536 y \$710 respectivamente).

Paquete com.unir.vista (La Vista): Contiene la clase InterfazCalculadora, desarrollada con la biblioteca Java Swing. Se encarga exclusivamente de la interfaz gráfica de usuario (GUI), mostrando el formulario de entrada, los botones de acción y una tabla (JTable) para visualizar los registros.

Paquete com.unir.main (El Controlador): Contiene la clase Principal, que es el punto de entrada del programa. En esta sección se instancian los objetos del modelo a partir de los datos recolectados en la vista. Adicionalmente, se implementó una estructura de datos estática ArrayList<ServicioPublico> que actúa como memoria temporal para guardar el historial de todos los servicios calculados durante la sesión, permitiendo generar un reporte consolidado final.

2. Historias de Usuario

Basado en los requerimientos del cliente UNIR S.A. y utilizando la estructura estándar (cómo, quiero, para qué), se definieron las siguientes historias funcionales:

HU01 - Cálculo de Tarifa: Como usuario, quiero ingresar el consumo numérico y seleccionar el tipo de servicio (Agua, Energía o Gas) para que el sistema calcule automáticamente el valor exacto a pagar según la tarifa parametrizada.

HU02 - Interfaz Gráfica: Como usuario, quiero interactuar con una interfaz visual que contenga campos de texto y listas desplegables para que la entrada de mis datos sea intuitiva y amigable.

HU03 - Validación de Datos: Como administrador del sistema, quiero que la aplicación valide internamente las entradas de texto para garantizar que el campo "consumo" acepte únicamente valores numéricos, evitando caídas del programa.

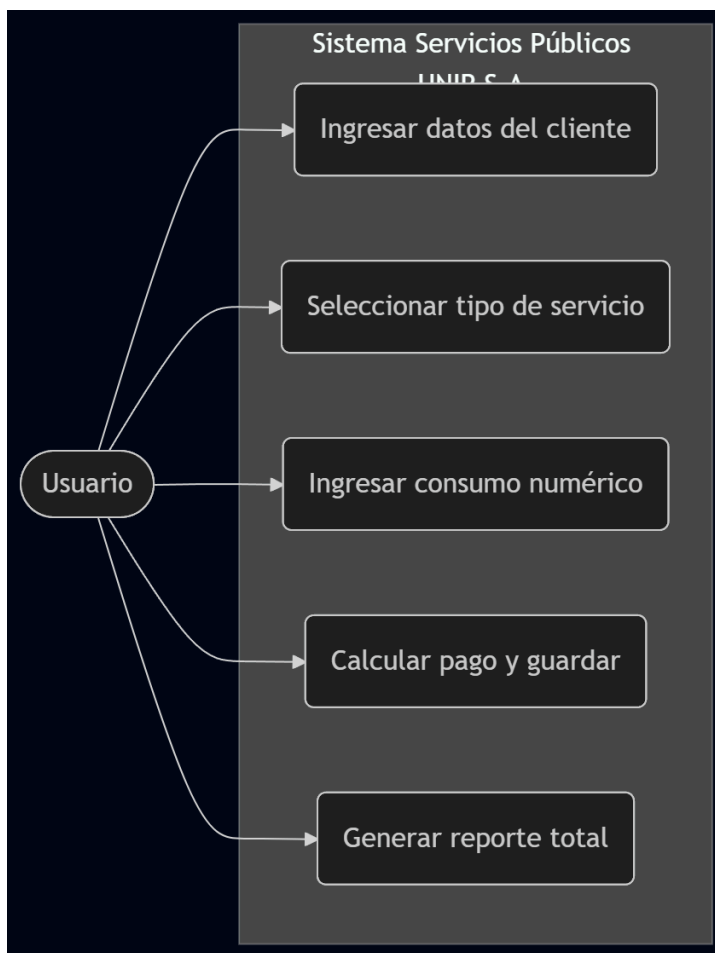
HU04 - Historial de Servicios: Como usuario, quiero poder guardar cada cálculo mediante un botón para mantener un registro visual en una tabla de todas las facturas procesadas en la sesión.

HU05 - Reporte Total: Como usuario, quiero presionar el botón "Generar" para visualizar un reporte emergente con el desglose y la suma total de todos los servicios guardados en el registro.

3. Diagramas Elaborados (UML)

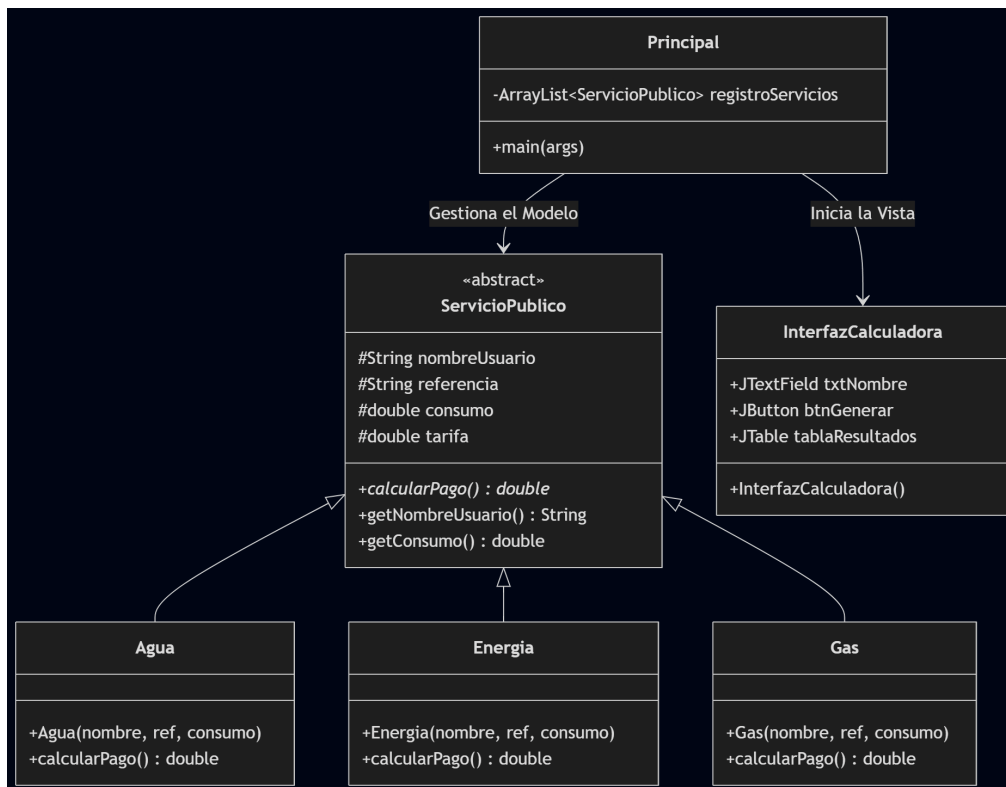
3.1 Diagrama de Casos de Uso

Descripción: Este diagrama ilustra la interacción directa entre el actor (Usuario) y el sistema de UNIR S.A. Representa los límites del sistema y las cinco acciones principales que el usuario puede ejecutar: ingresar datos, seleccionar el servicio, digitar el consumo, calcular/guardar el pago y generar el reporte final.



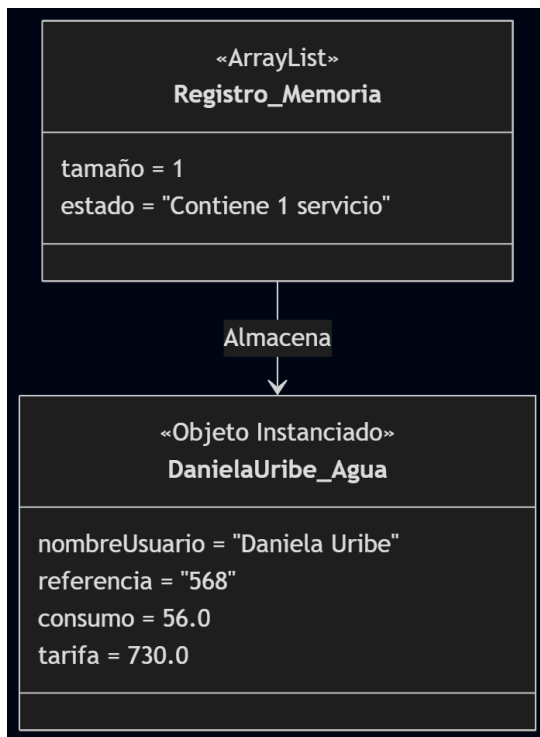
3.2 Diagrama de Clases

Descripción: Muestra la estructura estática del sistema y las relaciones de POO aplicadas. Se observa la clase abstracta `ServicioPublico` en el centro, de la cual heredan (flechas de generalización) las clases `Agua`, `Energía` y `Gas`. También ilustra cómo la clase `Principal` actúa como orquestador, gestionando una colección (`ArrayList`) del modelo y comunicándose con la `InterfazCalculadora`.



3.3 Diagrama de Objetos

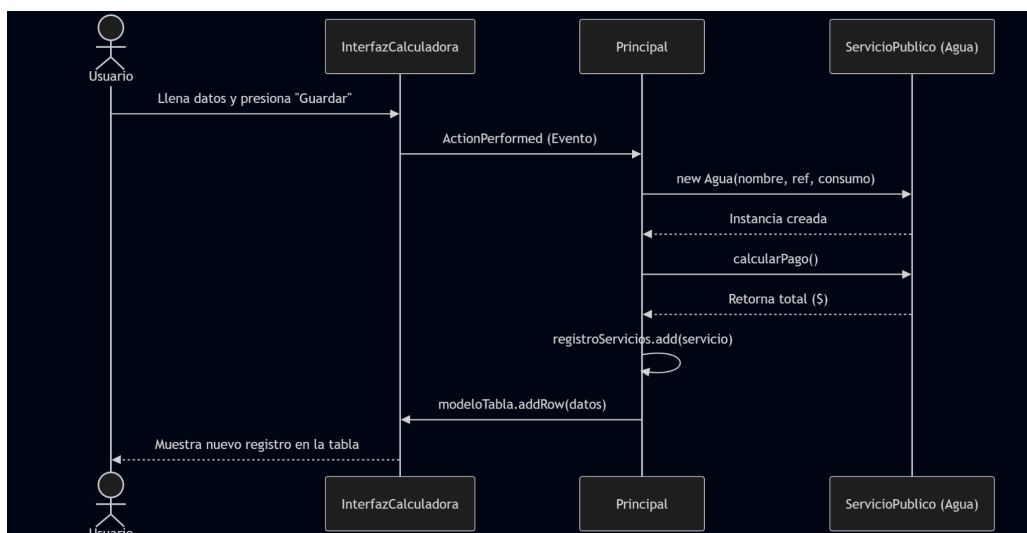
Descripción: Representa una instantánea (snapshot) del sistema en tiempo de ejecución. En este caso, ilustra la instanciación de un objeto de la clase `Agua` con los datos del caso de prueba (Usuario: Daniela Uribe, Ref: 568, Consumo: 56). Además, muestra cómo este objeto queda almacenado en la memoria temporal del `ArrayList` del registro.



3.4 Diagrama de secuencia

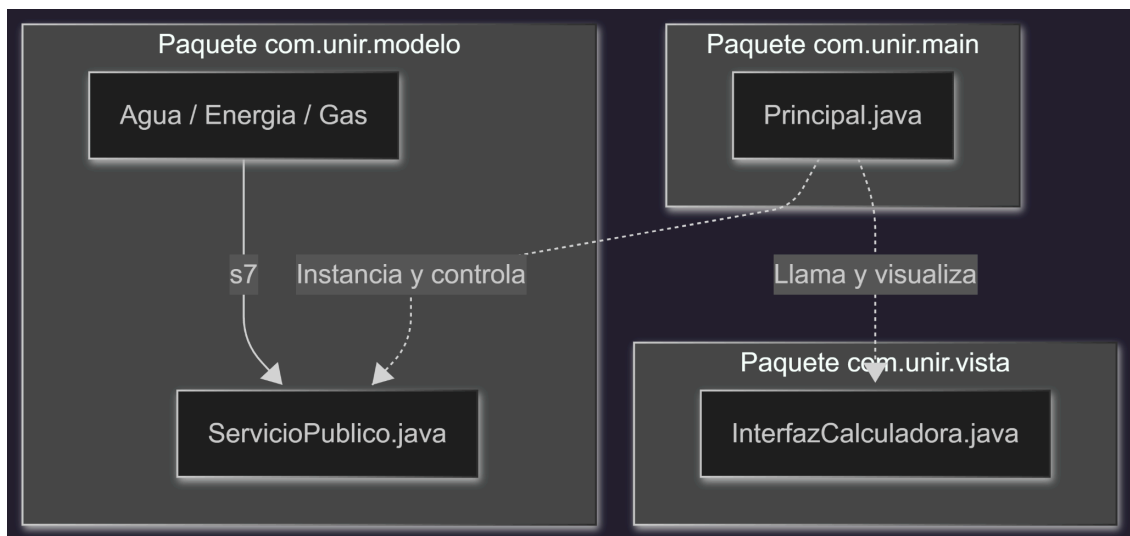
Descripción: Modela el comportamiento dinámico y cronológico del sistema.

Describe el flujo de mensajes cuando el usuario interactúa con la opción "Guardar". Muestra la ruta desde la Vista (captura de evento), pasando por el Controlador (instanciación), la llamada al Modelo para ejecutar `calcularPago()`, el almacenamiento en el `ArrayList` y el retorno visual a la tabla de la interfaz.



3.5 Diagrama de Componentes

Descripción: Proporciona una vista arquitectónica de alto nivel, mostrando cómo el código fuente está organizado lógicamente en paquetes. Evidencia la separación de responsabilidades: com.unir.vista maneja la presentación, com.unir.modelo contiene la lógica abstracta y com.unir.main contiene el núcleo ejecutable que vincula ambas partes.



C. Conclusiones

Aplicación de la Programación Orientada a Objetos (POO): El desarrollo de este caso de estudio permitió evidenciar la utilidad práctica de los pilares de la POO. A través de la herencia y el polimorfismo, se logró crear un sistema escalable donde una clase abstracta (ServicioPublico) define el comportamiento general, permitiendo que clases específicas (Agua, Energía, Gas) implementen sus propias reglas de cálculo de tarifas, lo que facilita agregar nuevos servicios en el futuro sin alterar la lógica principal.

Importancia de la Arquitectura MVC: Separar el proyecto en Modelo, Vista y Controlador resultó fundamental para mantener el código organizado. Esta práctica no solo facilitó la solución de requerimientos durante la fase de codificación, sino que permitió actualizar la interfaz gráfica de usuario y agregar un historial de registros en memoria (ArrayList) sin necesidad de reescribir las fórmulas matemáticas subyacentes.

Valor del Modelado UML: La elaboración de los diagramas (Casos de Uso, Clases, Objetos, Secuencia y Componentes) demostró ser una herramienta indispensable para la documentación del software. Estos modelos permiten traducir el código técnico a un lenguaje visual y estandarizado, facilitando la identificación de los requisitos del usuario, la comprensión de las interacciones internas y el flujo de la información dentro del sistema UNIR S.A.

D. Bibliografía

Piattini Velthuis, M. G. y Garzás Parra, J. (2015). Fábricas de software: experiencias, tecnologías y organización (Capítulo 7). RA-MA Editorial.

TecMD Fundación Politécnico Minuto de Dios. (2021). Estudio de Caso: Sistema Servicios Públicos UNIR S.A. [Documento PDF]. Programación y desarrollo de software.